Лабораторна робота №8

Тема: Використання асоціативних масивів (словників) для обробки даних.

Мета: Опанувати практичні навички роботи зі словниками (асоціативними масивами) в Python: їх створення, заповнення, обхід, сортування та використання для розв'язання задач текстового аналізу й кодування даних.

ОСНОВНІ ТЕОРЕТИЧНІ ВІДОМОСТІ

Словник (dict) в Python – це невпорядкована колекція пар ключ-значення, де кожен ключ є унікальним, а значення можуть повторюватися і бути будь-якого типу даних.

Створення словника:

```
d = {'China': 'Beijing', 'Canada': 'Ottawa'}
# за допомогою конструктора dict()
d = dict(x=1, y=2)
```

Основні методи та операції:

`d[key]` - доступ до значення за ключем (викликає помилку, якщо ключа немає) або метод d.get(key).

`d.items()` – повертає представлення пар (ключ, значення).

`d.keys()` – повертає ключі словника.

`d.values()` – повертає значення словника.

Сортування словників.

Оскільки словники за своєю природою невпорядковані, для їх сортування використовують вбудовану функцію `sorted()`, яка повертає відсортований список кортежів (ключ, значення). Для вибору критерію сортування (наприклад, за значеннями замість ключів) використовують параметр key разом із лямбда-функцією.

```
# Сортування за значеннями за спаданням
sorted_d = dict(
sorted(d.items(), key=lambda item: item[1],
reverse=True))
```

Обхід словників.

Обхід словників можна здійснювати трьома способами:

1. Обхід за ключами.

За замовчуванням або за допомогою методу `.keys()` цикл перебирає всі ключі словника:

```
d = {'x': 1, 'y': 2, 'z': 3}
for key in d: # або for key in d.keys():
   print(key, d[key])
```

2. Обхід за значеннями.

За допомогою методу `.values()` можна отримати доступ безпосередньо до значень словника.

```
for value in d.values():
    print(value)
```

3. Обхід за парами («ключ-значення»).

За допомогою методу .items() можна одночасно отримувати і ключ, і значення під час ітерації.

```
for key, value in d.items():
    print(f'Ключ: {key}, Значення: {value}')
```

Приклад використання словника – «Азбука Морзе».

Азбука Морзе – це спосіб кодування символів за допомогою коротких і довгих сигналів («крапок» . та «тире» −). Для реалізації такого перетворення в програмуванні ідеально підходить словник, де ключем виступає літра або цифра, а значенням – її графічне представлення в Морзе.

Приклад словника для базових символів:

```
morse_dict = {
    'A': '.-',
    'B': '-...',
    'C': '-.-.',
    'D': '-..',
    'E': '.',
    'F': '..-.',
    'G': '--.',
    'H': '....',
    'I': '..',
    'J': '.---',
    'K': '-.-',
    'L': '.-..',
    'M': '--',
    'N': '-.',
    'O': '---',
    'P': '.--.',
    'Q': '--.-',
    'R': '.-.',
    'S': '...',
    'T': '-',
    'U': '..-',
    'V': '...-',
    'W': '.--',
    'X': '-..-',
    'Y': '-.--',
    'Z': '--..',
```

```
    'S': '...',
}
```

Приклад 1. Виведення коду Морзе для окремої букви (з обробкою регістру)

Умова: Використати словник для виведення представлення букви, яку вводить користувач, у символах азбуки Морзе. Передбачити обробку малих і великих букв.

Вхідні дані: `D, f`

*Вихідні дані***:** `-.., ..-.`

Введення букви та приведення до верхнього регістру для обробки малих і великих літер

```
letter = input().strip().upper()

if letter in morse_dict:
  print(morse_dict[letter])
else:
  print('Символ відсутній у базі')
```

Приклад 2. Кодування цілого слова азбукою Морзе

Умова: Створити програму для кодування слова азбукою Морзе.

Вхідні дані: `SOS`

Вихідні дані: `...---...`

Код розв'язання:

```
morse_dict = {…}  # розширений словник для всіх букв

word = input().strip().upper()
encoded_word = []

for char in word:
  if char in morse_dict:
    encoded_word.append(morse_dict[char])

# Об'єднання елементів у рядок (можна додати пропуски між буквами за потреби)
print(''.join(encoded_word))
```

ЗАВДАННЯ ДЛЯ САМОСТІЙНОЇ РОБОТИ

Рівень І (Початковий)

1. **Підрахунок символів у рядку.** Напишіть програму для створення словника із введеного рядка символів для підрахунку кількості символів.

Вхідні дані: `Lorem ipsum dolor sit amet`

Вихідні дані: `{'L': 1, 'o': 3, 'r': 2, 'e': 2, 'm': 3, ' ': 4, 'i': 2, 'p': 1, 's': 2, 'u': 1, 'd': 1, 'l': 1, 't': 2, 'a': 1}`

2. **Перетворення одиниць вимірювання:** Напишіть програму, яка здійснює перетворення з однієї одиниці вимірювання довжини в інші (милі, ярди, фути, дюйми, кілометри, метри, сантиметри, міліметри) за допомогою словника коефіцієнтів.

Вхідні дані: `15.5 foot in mile`

Вихідні дані: `2.94e-03`

Рівень II (Середній)

1. **Сортування словника за значеннями (за зростанням):** Словник зберігає пари ключ-значення у вигляді «країна: столиця». Проведіть сортування за назвами столиць.

Вихідні дані: `[('China', 'Beijing'), ('Denmark', 'Copenhagen'), ('Ukraine', 'Kyiv'), ('Canada', 'Ottawa'), ('France', 'Paris')]`

2. **Сортування словника за значеннями у зворотному порядку (за спаданням):** Сортування за назвами столиць у порядку, протилежному алфавітному.

Вихідні дані: `[('France', 'Paris'), ('Canada', 'Ottawa'), ('Ukraine', 'Kyiv'), ('Denmark', 'Copenhagen'), ('China', 'Beijing')]`

3. **Аналіз карпатських вершин:** Дано словник, у якому ключами є назви 10 карпатських вершин, а значеннями – їх висота у метрах. Виведіть висоту трьох найвищих гір.

Вихідні дані:

```
Hoverla: 2061 m
Brebenescul: 2032 m
Pip Ivan Chernogirsky (Chorna gora): 2028 m
```

4. **Пошук міст за країнами:** За введеними скороченими назвами країн та міст виведіть для заданих міст їхні країни.

Вхідні дані: Місто `Seattle`

Вихідні дані: `USA`

Рівень III (Високий / Складні алгоритми)

1. **Операції над словниками (спільні ключі та різниця):** Дано два словники з символьними ключами та цілочисловими значеннями (`a = {'x' : 1, 'y' : 2, 'z' : 3},b = {'w' : 10, 'x' : 11, 'y' : 2}`).

Надрукуйте ключі, які присутні в першому словнику, але відсутні в другому, в одному рядку через пропуск.

Вихідні дані: `z`

2. **Частота слів у тексті:** Напишіть програму для обчислення частоти слів у введеному рядку. Виведення має здійснюватися у порядку, що зворотний буквенно-цифровому порядку (реверсивний сортований вивід).

Вхідні дані: `The five boxing wizards jump quickly`

3. **Статистика числової послідовності:** На вхід подається послідовність чисел від 1 до 9, що закінчується нулем (до 100 000 чисел). Підрахуйте за допомогою словника кількість одиниць, двійок, трійок тощо.

Вхідні дані: `1 1 4 1 5 8 6 3 5 1 0`

Вихідні дані: `4 0 1 1 2 1 0 1 0`

Контрольні запитання

1. Чим відрізняється словник (dict) від списку (list) у Python?
2. Які типи даних можуть бути ключами у словнику? Чи може ключ бути списком?
3. Яким чином можна безпечно отримати значення за ключем, уникнувши помилки `KeyError`?
4. Для чого використовується лямбда-функція (`lambda`) у парі з методом `sorted()` при роботі зі словниками?